

COMP3069 Computer Graphics

Assessment 2

For assessment 2, your task is to program a 2D rasteriser. You are given 1.5 hours (coding time) to complete this coding assessment. **The reading time and the submission time** are also given. The program is not expected to be long and will be in one file. You will need to write C++ code, which will be split into functions.

You will **need to implement**

1. ClearColourBuffer(int r, int g, int b)
2. AssembleTriangles(float* verts, int n_verts, int* n_tris)
3. TransformToViewport(int width, int height, triangle* tri)
4. ComputeBarycentricCoordinates(float x, float y, triangle t, float& alpha, float& beta, float& gamma)
5. ShadeFragment(float alpha, float beta, float gamma, triangle t, int& r, int& g, int& b)

You are **NOT allowed to add or remove** any variables in the above functions.

You will need to submit, to Moodle, a copy of the file containing the source code of your implementation. You only need to submit “COMP3069_ASSESSMENT_2.cpp”. Only submit one single file. When you test your program in the Visual Studio solution before submitting, make sure that it runs on your PCs or on the PCs in the lab.

Setting up

Download the COMP3069_ASSESSMENT_2 zipped directory from Moodle with the link provided. Extract the files. The solution does not need any of the GLFW or GLAD files from the lab exercises, is self-contained, and just uses SDL (a cross-platform development library), so you can put the directory anywhere. Open up “COMP3069_ASSESSMENT_2.sln” and click on the Run Button and you should see an empty and black window.

Get to know the files

At the top of COMP3069_ASSESSMENT_2.cpp you will see the following vertex array:

```
11     float vertices[] =
12     {
13         //pos
14         -.75f, .5f, 0.f,           //tl
15         .75f, .5f, 0.f,           //tr
16         .0f, .0f, 0.f,           //b
17     };
```

The vertex attributes are the vertex positions of a **triangle** in NDC space, (-1 to 1 in X, Y and Z directions). Since the rasteriser is for 2D, the z component should be 0 for all vertices.

You **will be requested to change** the attributes for the triangle in the `vertices[]` to a new set in this test.

Underneath, you will see the empty functions that you will need to implement:

```
20 void ClearColourBuffer(int r, int g, int b)
21 {
22 }
23 triangle* AssembleTriangles(float* verts, int n_verts, int* n_tris)
24 {
25     return NULL;
26 }
27 void TransformToViewport(int width, int height, triangle* tri)
28 {
29 }
30 void ComputeBarycentricCoordinates(float x, float y, triangle t, float& alpha, float& beta, float& gamma)
31 {
32 }
33 void ShadeFragment(float alpha, float beta, float gamma, triangle t, int& r, int& g, int& b)
34 {
35 }
```

At the bottom you will find the `main()` function:

```
39 int main()
40 {
41     COMP3069StartSDL();
42
43     while (1)
44     {
45         if (COMP3069PressedEscape())
46             break;
47
48         COMP3069DisplayColourBuffer();
49     }
50
51     COMP3069StopSDL();
52
53     return EXIT_SUCCESS;
54 }
```

The meanings of the `COMP3069***` functions are as follows.

1. `COMP3069StartSDL()` initialises SDL and creates a window.
2. `COMP3069PressedEscape()` returns true if the user presses the escape key.
3. `COMP3069DisplayColourBuffer()` copies whatever is in the colour buffer to the SDL window.
4. `COMP3069StopSDL()` closes SDL.

You **do NOT need to change** any function implementation or function call of any function starting with `COMP3069`.

Completing the assessment

It should be obvious that the order of the functions needs to be called to rasterise a triangle.

The Colour Buffer

The colour buffer is a 2D array of `vec3` elements. It is as large as the display `PIXEL_W x PIXEL_H`. **The `PIXEL_W` and `PIXEL_H` and `vec3` struct are defined in "DoNotModify.h"**, which as the name suggests, you **do NOT need to and should not modify**. `vec3` has x, y, and z components. It is used by the vertex position XYZ components and also by the colour RGB components. For the colour buffer, the x corresponds to the R, y to G, and z to B values, which will be displayed at a given pixel.

In the `main()` function, you will find `COMP3069DisplayColourBuffer()`. It is defined in "DoNotModify.h", and it copies each pixel colour from the colour buffer to the SDL display. Note that SDL uses [0, 1, ..., 255] for the RGB components of a colour.

To start, in the `main()` function, try to set the colour of a given pixel in the `colour_buffer` and verify that it is copied to the display when you run the program. **Implement the function** `void ClearColourBuffer(int r, int g, int b)`, which clears the whole colour buffer to a colour specified by the `r`, `g`, `b` arguments.

Triangles

Next, **implement** `AssembleTriangles()`

```
triangle* AssembleTriangles(float* verts, int n_verts, int* n_tris)
```

where `verts` is the array of vertex attributes, `n_verts` is the number of vertices, `n_tris` will hold the number of triangles. This function should allocate memory for `triangles`, copy and locate the vertex attributes to each triangle vertex position, update the value of `n_tris` and return **all** `triangles`, with necessary translation and scaling transformations, **in correct locations**.

Viewport transformation

Next, **implement** `TransformToViewport()`

```
void TransformToViewport(int width, int height, triangle* tri)
```

This function should transform the vertex positions from NDC to pixel coordinates.

Barycentric Coordinates

We need to calculate the barycentric coordinates and use them to test if a pixel is inside a triangle. Implement the function `ComputeBarycentricCoordinates()`

```
void ComputeBarycentricCoordinates(float x, float y, triangle t, float& alpha, float& beta, float& gamma)
```

where x and y are the current pixel coordinate, t is the triangle to be tested, α , β and γ will store the barycentric coordinates.

Fragment Shader

For this coursework, the fragment shader should return two different colours for the triangle and the rectangle, respectively

```
void ShadeFragment(float alpha, float beta, float gamma, triangle t, int& r, int& g, int& b)
```

Please **ignore** the barycentric coordinates α , β , and γ and do **not** assign any values to them.

Rasterisation

You need to implement the rasteriser. This is the easy part and is done automatically once you have everything else set up.

SDL screen space

When you first get your rasteriser working, you may probably notice that the triangles are drawn upside down. This is because in SDL the screen space is



Please provide a correct way to fix this upside-down issue.

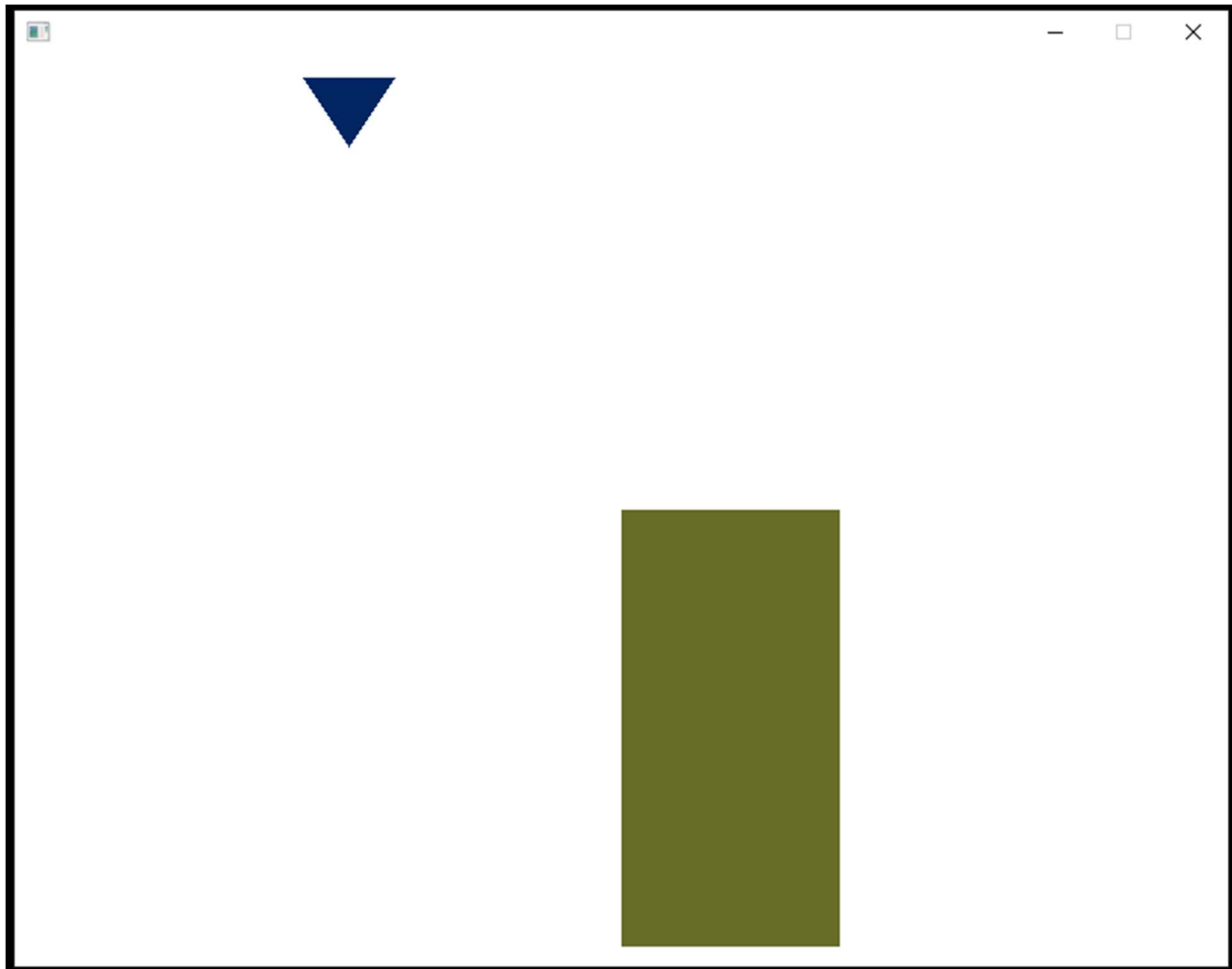
More tasks (requirements)

1. Please change the attributes for the triangle specified in the vertices[] to:

```
11 float vertices[] =  
12 {  
13     //pos  
14     -.5f, .75f, 0.f,      //tl  
15     .5f, .75f, 0.f,      //tr  
16     .0f, .0f, 0.f,      //b  
17 };
```

2. Please **add** a rectangle in NDC space with sides parallel to the horizontal and vertical axes and a width-to-height ratio of 1:2. You will need to specify the coordinates for its vertices in the model space.
3. Please allocate top-left vertex of this rectangle at the centre of the screen space.
4. Please move (**translate**) the triangle to the top-left corner of the screen.
5. You may need to **scale** up or down the rectangle and triangle so that both of them should all be **fully seen** on the screen, their **aspect ratios are the same** as those in the NDC space, and they are **NOT overlapping**.
6. Please do **NOT make additional transformations** of both rectangle and triangle.
7. Please use your **own chosen different colours** to fully colour the rectangle and triangle, respectively.
8. It is expected that **no student** will have the sizes and/or colours of the rectangle and triangle to be the same as those of any other student.
9. Please do **NOT create any additional functions**.

Here is a sample output meeting the above requirements:



Marking Scheme

The following marking scheme will be used.

Test #	Test	Marks	Completion
1	Are the attributes for the triangle specified in the vertices[] changed?	3	Correct specification (9)
2	Are the attributes for the rectangle correctly added and specified in the vertices[]?	6	
3	Is <code>ClearColourBuffer()</code> implemented correctly?	6	Clear screen (6)
4	Is <code>AssembleTriangles()</code> implemented correctly and completely?	10	Triangle rasterized (35)
5	Is <code>TransformToViewport()</code> implemented correctly?	10	
6	Is <code>ComputeBarycentricCoordinates()</code> implemented correctly?	10	
7	Is the triangle the right way up and displayed correctly?	5	

8	Can the triangle and rectangle colours be correctly specified in <code>ShadeFragment()</code>	10	Shader (25)
9	Is the additional rectangle rendered and displayed on the screen correctly?	5	
10	Are the colours of the rectangle and triangle unique and different from other students' work?	5	
11	Are the colours of the rectangle and triangle different from each other?	5	
12	Is the top-left vertex of the rectangle at the centre of the screen?	5	Translation (10)
13	Is the triangle at the top-left corner of the screen?	5	
14	Is the rectangle scaled correctly?	5	Scaling (10)
15	Is the triangle scaled correctly?	5	
16	Are the rectangle and triangle fully seen on the screen and are NOT overlapping?	5	Display (5)
Total		100	